



Math & Deep Learning

Redes concolucionales y análisis de Fourier

Índice

01. Introducción	pág. 03
01. Yann LeCun	pág. 04
02. Redes Neuronales Convolucionales	pág. 06
01. Perceptrón multicapa entre matrices (imágenes)	pág. 07
02. Invarianza local por traslación	pág. 09
03. Análisis de Fourier y convoluciones	pág. 11
01. La transformada de Fourier	pág. 12
02. Convoluciones en análisis de fourier	pág. 15
04. Convoluciones y procesamiento de imágenes	pág. 17
01. Convoluciones 1-dimensionales	pág. 17

02. Convoluciones 2-dimensionales	—
pág. 19	
03. Detección de objetos	_____
	pág. 20
04. Evaluación de la detección de objetos	
pág. 21	
05. Knowledge Check	_____
	pág. 23

01 Introducción

Bienvenidos al curso Math & Deep Learning. En él buscamos introducir al estudiante en los fundamentos matemáticos y conceptuales que sustentan las arquitecturas modernas de redes neuronales. Más allá de la aplicación práctica, se enfatiza el entendimiento riguroso de cómo las estructuras algebraicas, los procesos estocásticos y las funciones no lineales dan forma al aprendizaje profundo. El estudiante no solo obtendrá destrezas técnicas, sino que comprenda la matemática que hace posible el aprendizaje profundo, con el fin de construir y adaptar modelos más sólidos, interpretables y eficientes.

A lo largo del programa estudiaremos redes densas, convolucionales y recurrentes; vinculándolas con herramientas analíticas como el análisis de Fourier y los procesos estocásticos. También abordaremos el entrenamiento y la optimización de redes, explorando la paralelización y técnicas de tuning, así como la aplicación de auto-encoders en conjunto con la teoría de valores extremos. Finalmente, exploraremos los modelos de Generative AI y GANs, conectando la matemática con los desarrollos más recientes en inteligencia artificial. Los módulos del curso son los siguientes:

1. Redes densas & otras funciones no lineales
2. Redes convolucionales (CNN) & Análisis de Fourier

3. Redes recurrentes (RNN, LSTM, GRU) & Procesos estocásticos
4. Optimización en redes densas & GPU
5. Autoencoders (densas & recurrentes) v.s. Teoría de los Valores Extremos
6. Generative Adversarial Networks (GAN) & Teoría de Juegos

🔗 El repositorio de los casos prácticos se puede encontrar en el siguiente [link](#)

Yann LeCun



Estudió ingeniería eléctrica en ESIEE París y un doctorado en informática en la Université Pierre et Marie Curie. Actualmente es el Chief AI Scientist en Meta y es uno de los científicos más influyentes en el desarrollo y la implementación de las redes neuronales profundas.

Como veremos en este curso, las convoluciones son un objeto matemático que permite operar parejas de funciones y tiene sus raíces en el trabajo de Fourier sobre las representaciones trigonométricas de funciones para resolver la ecuación diferencial del calor. Estos objetos se habían utilizado anteriormente en redes neuronales para clasificar dígitos, sin embargo los parámetros que se utilizaron en los trabajos previos a LeCun estaban fijos y fueron elegidos manualmente. Posiblemente él fue la primera persona en utilizar el algoritmo de Backpropagation para entrenar los parámetros utilizando la supervisión de los datos. Esta idea osada de entrenar no solo funciones densas sino otros objetos es quizás el corazón detrás de los recientes avances en inteligencia artificial.

02 Redes Neuronales Convolucionales

Hemos ejemplificado cómo al utilizar más capas en una red neuronal (y neuronas) es posible aumentar la capacidad expresiva de una red neuronal. No es difícil convencerse (y demostrar matemáticamente) que si se aumentamos la cantidad de capas, es posible aproximar cualquier relación funcional entre las entradas x_i de una base de datos y su etiquetado z_i . No obstante, aumentar la cantidad de capas y neuronas sin control puede llegar a ser problemático, pues rápidamente podríamos generar el fenómeno de sobreajuste en nuestras predicciones, es decir, que nuestra red neuronal se entrene muy bien en la base de datos S , sin embargo no tiene buena predicción en nuevos ejemplos.

Para remediarlo se consideran redes neuronales que por un lado son profundas (i.e. tienen varias capas y muchas neuronas), y por otro lado, no todas las neuronas deben estar conectadas entre sí. Un ejemplo de tales redes son las Redes Neuronales Convolucionales las cuales son el state of the art de los problemas de clasificación.

Las convoluciones son una de las herramientas más útiles en procesamiento de señales. Clásicamente se utiliza para reducir el ruido de una señal, sin embargo a raíz del trabajo de Yann LeCun, esta operación entre funciones se ha utilizado para construir arquitecturas de redes

neuronales capaces de entrenarse exitosamente para tareas relacionadas con imágenes.

Vale la pena mencionar que la operación llamada convolución por los expertos en machine learning no coincide exactamente con la definición matemática estricta, sin embargo las diferencias son nimias.

Perceptrón multicapa entre matrices (imágenes)

Recordemos que las redes neuronales totalmente conexas también conocidas como perceptrón multicapa. La ecuación que utilizamos para el perceptrón es:

$$w_j = \rho(\langle m_j, x \rangle + b_j) \quad (02.1)$$

Donde por supuesto, la capa inicial del perceptrón es x y la siguiente es w .

En este módulo hablaremos de imágenes y por tanto nos conviene pensar en nuestras capas como arreglos matriciales para poder asegurar que los pesos están considerando la estructura interna de las imágenes.

Para el caso cuando nuestra primera capa es una matriz $(x_{i,j})_{i,j \leq n}$ y la siguiente es una matriz $(w_{r,s})_{r,s \leq m}$, entonces un perceptrón totalmente conexo entre ellas se puede escribir de la siguiente manera:

$$w_{r,s} = \rho \left(\sum_i \sum_j (\beta_{r,s,i,j} x_{i,j}) + b_{r,s} \right) \quad (02.2)$$

El arreglo (conocido como 4-tensor) $\beta \in \mathbb{R}^{n \times n \times m \times m}$ contiene demasiados parámetros, pero las capas convolucionales reducirán la cantidad de parámetros con el objetivo de mejorar la calidad estadística de los modelos.

Reescribamos la ecuación 02.2 utilizando el siguiente cambio de variables

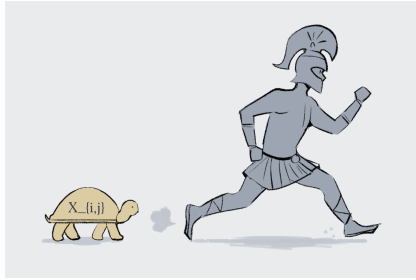
$$i = r + I, \quad j = s + J \quad (02.3)$$

para obtener una nueva versión de la ecuación 02.2:

$$w_{r,s} = \rho \left(\sum_I \sum_J (\beta_{r,s,I,J} x_{r+I,s+J}) + b_{r,s} \right) \quad (02.4)$$

La nueva ecuación puede interpretarse de la siguiente manera: una variable latente $w_{r,s}$ es igual a una suma ponderada de todos los píxeles que están al rededor del píxel en la misma posición de la primera matriz.

Exercise 01.1. Dejamos al lector dibujar qué significa el cambio de variable anterior en el caso de suponer que tenemos un perceptrón entre vectores y no entre matrices.



Invarianza local por traslación

Una primera manera en la que podremos disminuir la cantidad de parámetros libres en una capa densa es pedir que los pesos asociados a una sección de la imagen no varíen dependiendo de la posición en la imagen inicial. Esto permitiría reducir el 4-tensor a un 2-tensor con pesos:

$$\beta_{r,s,I,J} = \beta_{I,J} \quad (02.5)$$

Entonces podemos reescribir la ecuación del perceptrón de la siguiente manera:

$$w_{r,s} = \rho \left(\sum_I \sum_J (\beta_{I,J} x_{r+I,s+J}) + b \right) \quad (02.6)$$

Esto implica que si tenemos dos píxeles distintos de la imagen original que comparten las mismas vecindades, entonces el valor de la variable latente será igual. Digamos $x_{r'+I,s'+J} = x_{r+I,s+J}$ para todos los I, J , entonces

$$w_{r,s} = \rho \left(\sum_I \sum_J (\beta_{I,J} x_{r+I,s+J}) + b \right) = \rho \left(\sum_I \sum_J (\beta_{I,J} x_{r'+I,s'+J}) + b \right)$$

El grave problema de la ecuación anterior es que requiere la igualdad $x_{r'+I,s'+J} = x_{r+I,s+J}$ para una vecindad completa al rededor de los pixeles x y x_r y x'_r , lo cuál podría ser restrictivo. En su lugar vamos a suponer que existe una lejanía entre ellos $p > 0$, tal que $\beta_{I,J} = 0$ cuando $|I|, |J| > p$.

Lo anterior significa que es posible reescribir la ecuación del perceptrón de la siguiente manera:

$$w_{r,s} = \rho \left(\sum_{-p < I < p} \sum_{-p < J < p} (\beta_{I,J} x_{r+I,s+J}) + b \right) \quad (02.7)$$

Example 02.1. Supongamos que tenemos una imagen $(x_{i,j})$. Sea $z_{r,s}$ el resultado de reducir la resolución de $(x_{i,j})$ promediando los pixeles de X al rededor de $x_{r,s}$. Es decir,

$$z_{r,s} = \begin{cases} \beta_{-1,-1}x_{r-1,s-1} & +\beta_{-1,0}x_{r-1,s} & +\beta_{-1,1}x_{r-1,s+1} \\ +\beta_{0,-1}x_{r,s-1} & +\beta_{0,0}x_{r,s} & +\beta_{0,1}x_{r,s+1} \\ +\beta_{1,-1}x_{r+1,s-1} & +\beta_{1,0}x_{r+1,s} & +\beta_{1,1}x_{r+1,s+1} \end{cases} \quad (02.8)$$

Donde los pesos de $(\beta_{I,J})_{|I|,|J|\leq 1}$, son todos $\frac{1}{9}$.

03 Análisis de Fourier y convoluciones

El análisis de Fourier es una rama del análisis matemático cuyo objetivo es la representación de funciones (o señales), $f(t)$ como una suma ponderada de senos y cosenos de distintas frecuencias. Esta representación permite pasar del dominio del tiempo al dominio de la frecuencia. El cambio del dominio de una señal f , permite su descomposición en las frecuencias puras que la componen.

Por ejemplo, en el procesamiento de imágenes, los bordes, texturas y detalles finos de las imágenes corresponden a frecuencias altas, mientras que las superficies lisas o zonas uniformes corresponden con frecuencias bajas. Intuitivamente, aplicar un filtro para detectar los bordes de una imagen es la operación opuesta a de aplicar un filtro de suavizado de la imagen (blur).

Para una función integrable y periódica $f(t)$ de periodo T (es decir, $f(x + T) = f(x)$), la serie de Fourier asociada es:

$$S_N(t) = \frac{a_0}{2} + \sum_{n=1}^N (a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t)), \quad (03.1)$$

Donde $\omega_0 = \frac{2\pi}{T}$ y los coeficientes de Fourier se definen como:

$$\begin{aligned}a_0 &= \frac{2}{T} \int_{t_0}^{t_0+T} f(t) dt, \\a_n &= \frac{2}{T} \int_{t_0}^{t_0+T} f(t) \cos(n\omega_0 t) dt, \\b_n &= \frac{2}{T} \int_{t_0}^{t_0+T} f(t) \sin(n\omega_0 t) dt, \quad (n \geq 1).\end{aligned}$$

Cuando $N \rightarrow \infty$ en 03.1 se obtiene la aproximación de Fourier de f .

$$f(t) \sim \frac{a_0}{2} + \sum_{n=1}^{\infty} (a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t)). \quad (03.2)$$

O bien, en su forma compleja:

$$f(t) \sim \sum_{n=-\infty}^{\infty} c_n e^{in\omega_0 t}$$

donde los coeficientes c_n satisfacen

$$C_n = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) \cdot e^{-ind} dt$$

La transformada de Fourier

La Transformada de Fourier es una de las herramientas matemáticas más poderosas en la ciencia moderna. Jean-Baptiste Joseph Fourier (1768–1830) introdujo el concepto al estudiar la conducción del calor.



La Transformada de Fourier permite traducir el comportamiento de una función o señal en el tiempo a su representación en el dominio de la frecuencia. Esto significa que, en lugar de describir cómo varía la señal punto a punto en el tiempo, la representamos en términos de las frecuencias elementales que la componen. Cada frecuencia corresponde a una onda senoidal (real o compleja) con cierta amplitud y fase. La transformada nos indica cuánta energía o intensidad aporta cada una de esas ondas a la señal original.

En este nuevo dominio, las operaciones que en el tiempo son complicadas (como la convolución o el filtrado de señales) se vuelven más simples pues se transforman en multiplicaciones, lo que hace posible analizar y modificar las señales desde una perspectiva espectral.

Definition 01.1. Sea $f(t) \in L^1(\mathbb{R})$, su transformada de Fourier está

dada por:

$$\mathcal{L}\{f(t)\}(\omega) = \hat{f}(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t} dt \quad (03.3)$$

y la transformada inversa:

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega)e^{i\omega t} d\omega \quad (03.4)$$

Esta representación permite expresar una señal como una superposición de ondas sinusoidales.

Definition 01.2. Definimos la integral de convolución de dos funciones continuas f y g como la integral:

$$f * g(t) = \int_{-\infty}^{\infty} f(x)g(t-x)dx \quad (03.5)$$

Las principales propiedades de la transformada de Fourier son las siguientes:

- Linealidad: $\mathcal{L}\{af + bg\} = a\hat{f} + b\hat{g}$
- Desplazamiento en el tiempo: $\mathcal{L}\{f(t - t_0)\} = e^{-i\omega t_0}\hat{f}(\omega)$
- Desplazamiento en frecuencia: $\mathcal{L}\{e^{i\omega_0 t}f(t)\} = \hat{f}(\omega - \omega_0)$
- Escalamiento: $\mathcal{L}\{f(at)\} = \frac{1}{|a|}\hat{f}\left(\frac{\omega}{a}\right)$
- Convolución: $\mathcal{L}\{f * g\} = \hat{f}(\omega)\hat{g}(\omega)$
- Transformada de una derivada: $\mathcal{L}\left\{\frac{d^n f}{dt^n}\right\} = (i\omega)^n \hat{f}(\omega)$

Convoluciones en análisis de fourier

En esta sección incluimos algunas nociones de la teoría de convoluciones para funciones generales.

Definition 02.1. Si $x : \mathbb{R} \rightarrow \mathbb{R}^d$ y $u \in \mathbb{R}$, la distribución (en términos de Schwartz) de Dirac centrada en u , δ_u , es aquella que satisface la siguiente ecuación:

$$(u) = \int_{-\infty}^{\infty} x(v) \delta_{u-v} dv \quad (03.6)$$

Gracias a la ecuación anterior, es posible expresar cualquier función x como una "suma" de deltas de Dirac. Se utiliza esta fórmula para diagonalizar los operadores invariantes por traslación.

Supongamos que x es suficientemente continua para que la siguiente ecuación sea cierta:

$$\mathcal{L}(x(v) \delta_{u-v}) = \mathcal{L}\left(\int_{-\infty}^{\infty} x(v) \delta_{u-v}\right) = \int_{-\infty}^{\infty} x(x) \mathcal{L}(\delta_{u-v}) \quad (03.7)$$

Ahora para clasificar a $\mathcal{L}$ solo es necesario entender el impulso $\mathcal{L}(\delta_{u-v}) =: h(u-v)$, de esta manera recuperamos la definición clásica de la convolución:

$$\mathcal{L}(x(u-v)) = \int_{-\infty}^{\infty} x(v) h(u-v) dv = x \star h(u) \quad (03.8)$$

La pregunta inmediata es si estos operadores L se pueden diagonalizar.

La respuesta es sí en la base de Fourier.

Definamos la función

$$x_v(u) = e_v(u) = e^{iuv} \quad (03.9)$$

Es fácil verificar que

$$e_v \star h(u) = e^{iuv} \int_{-\infty}^{\infty} h(v) e^{-ivu} dv \quad (03.10)$$

La convolución de la que hablaremos en el siguiente capítulo no son precisamente las que se utilizan en el análisis de Fourier, sino una versión discreta y mucho más sencilla, que explica cómo se procesan las imágenes de manera muy explícita.

04 Convoluciones y procesamiento de imágenes

Convoluciones 1-dimensionales

El caso 1 dimensional de las convoluciones (discretas) nos podría ayudar a entender mejor cómo funcionan las convoluciones.

Definition 01.1. Convoluciones 1D. Sean $(x_t)_{1 \leq t \leq T}$ las observaciones de una serie de tiempo, si $\beta_{-\kappa}, \beta_{-\kappa+1}, \dots, \beta_0, \dots, \beta_{\kappa-1}, \beta_{\kappa} \in \mathbb{R}$ es una familia de pesos, definimos la convolución g entre $(x_t)_{1 \leq t \leq T}$ y $\beta = (\beta_i)_{-\kappa \leq i \leq \kappa}$ de la siguiente manera para cualquier $t \leq T$:

$$(x \star g)_r = \beta_{-\kappa} x_{r-\kappa} + \beta_{-\kappa+1} x_{r-\kappa+1} + \dots + \beta_{\kappa-1} x_{r+\kappa-1} + \beta_{\kappa} x_{r+\kappa} \quad (04.1)$$

O bien,

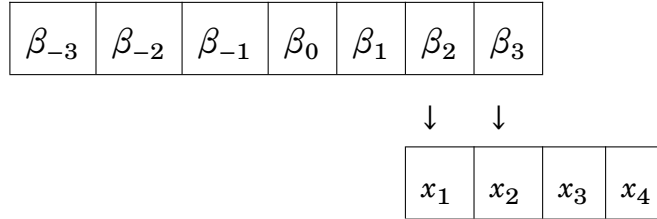
$$(x \star g)_r = \sum_I (\beta_I x_{r+I}) \quad (04.2)$$

Diremos que una convolución uno-dimensional entre x y β es de tamaño R cuando $r \leq R$.

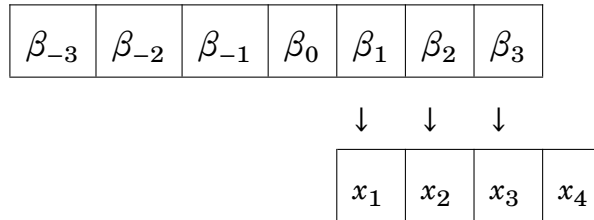
Example 01.1. Consideremos $\kappa = 3$ y $T = 4$ $\beta = (\beta_{-3}, \beta_{-2}, \dots, \beta_3)$.

Entonces:

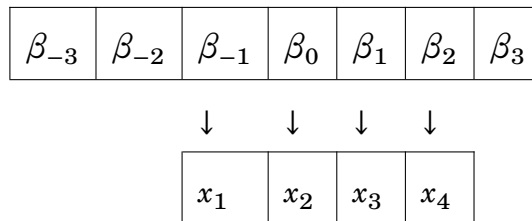
$$r = -1 \quad (x \star g)_{-1} = \beta_{-3}x_{-4} + \beta_{-2}x_{-3} + \beta_{-1}x_{-2} + \beta_0x_{-1} + \beta_1x_0 + \beta_2x_1 + \beta_3x_2.$$



$$r = 0 \quad (x \star g)_0 = \beta_{-3}x_{-3} + \beta_{-2}x_{-2} + \beta_{-1}x_{-1} + \beta_0x_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_3.$$



$$r = 2 \quad (x \star g)_2 = \beta_{-3}x_{-1} + \beta_{-2}x_0 + \beta_{-1}x_1 + \beta_0x_2 + \beta_1x_3 + \beta_2x_4 + \beta_3x_5.$$



Inspirados en la discusión anterior sobre los perceptrones multicapa notemos que el número de neuronas en la capa interna está fijo lo cual lo convierte en un hiperparámetro.

Example 01.2. Un caso particular es cuando $\beta_s = \frac{1}{2p+1}$ para todo $-p \leq s \leq p$. Aquí recuperamos el promedio móvil clásico.

Example 01.3. Otro caso particular es cuando $\beta_0 = 1$ y para todo $s \neq 0$ y $\beta_s = 0$; en ese caso recuperamos la serie $x_t = (x \star g)_t$. A este núcleo se le llama la delta de Dirac centrada en cero y se denota δ_0 .

Remark 01.4. A diferencia de las convoluciones, la operación de un promedio móvil no permite utilizar las observaciones posteriores a x_t , esta característica de las convoluciones es fundamental para construir las redes neuronales.

Convoluciones 2-dimensionales

Definition 02.1. Convoluciones 2D. Sean $x = (x_{i,j})_{1 \leq i,j \leq n}$ los pixeles asociados a una imagen, si $g : \{1, 2, \dots, K\}^2 \rightarrow \mathbb{R}$ es una función arbitraria, entonces definimos la convolución entre x y g de la siguiente manera:

$$(x \star g)_{r,s} = \sum_{I \leq N} \sum_{J \leq N} (g(I, J) x_{r+I, s+J}) \quad (04.3)$$

Remark 02.1. A la nueva matriz $(x \star g)_{i,j}$ las llamaremos el feature map mientras que la función g recibe el nombre de kernel.

Example 02.2. En el ejemplo de disminución de la resolución de la imagen X, 02.1, la nueva imagen Z es el feature map y el kernel $g : [-1, 0, 1]^2 \rightarrow \mathbb{R}$ corresponde a rodar al pixel $x_{r,s}$ con un cuadrado de 3×3 .

02.1 Propiedades de las convoluciones 2D

A continuación enunciaremos algunas propiedades indispensables para entender cómo funcionan las convoluciones y por qué funcionan bien para datos como las imágenes.

Proposition 02.3. Si x es una señal 2-dimensional y g, g' son dos kernel cualesquiera entonces:

a) $x \star g = g \star x$

b) $(x \star g) \star g' = x \star (g \star g')$

Detección de objetos

Hasta el momento hemos descrito tareas como la clasificación y la regresión para las cuales nuestras variables dependientes y son iguales a $\{-1, +1\}$ o están dentro de $\mathbb{R}$.

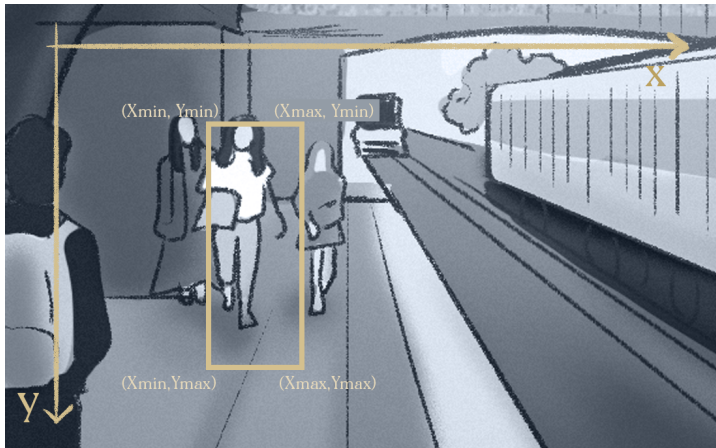
El problema de la detección de objetos requiere y 's que serán una mezcla de los dos casos anteriores.

Definition 03.1. Una base de datos supervisada y binaria para la detección de objetos en imágenes es un conjunto:

$$S = \left\{ (x_1, y_1), \dots, (x_N, y_N) \right\}$$

Donde:

1. $x_i \in \mathbb{R}^d$ corresponde a una manera de vectorizar las imágenes,
2. $y_i = (y_{i,c}, y_{X,min}, y_{X,max}, y_{Y,min}, y_{Y,max}) \in \{-1, +1\} \times [0, 1]^4$



Evaluación de la detección de objetos

Para evaluar este tipo de modelos es necesario construir una estadística que nos diga cuánto acierto produce una predicción.

Definition O4.1. El Índice de Jaccard. Si en $\{-1, +1\} \times [0, 1]^4$, los vectores

$$y = (y_{X,min}, y_{X,max}, y_{Y,min}, y_{Y,max})$$

$$y' = (y'_{X,min}, y'_{X,max}, y'_{Y,min}, y'_{Y,max})$$

representan solo las coordenadas (de las esquinas) de un rectángulo I_y

e $I_{y'}$, respectivamente. El índice de Jaccard de y y y' se define como el cociente entre la intersección de los pixeles de y y de y' , con la unión de sus pixeles:

$$J(y, y') = \frac{I_y \cap I_{y'}}{I_y \cup I_{y'}} \quad (04.4)$$

Donde I_y denota a todos los de pixeles dentro del rectángulo determinado por las coordenadas de y .

Remark 04.1. Observemos que el índice de Jaccard es cercano a 1 cuando y y y' son muy cercanos.

Knowledge Check

- Exercise 05.1.
1. En qué problema considera usted que se puede utilizar el algoritmo visto esta semana? Por favor, describa la base de datos que le gustaría utilizar e indique por qué este algoritmo es el más adecuado.
 2. Es posible utilizar convoluciones para el procesamiento de un tipo de datos que no sean imágenes?

BOURBAKI

ESCUELA DE MATEMÁTICAS

CURIOSIDAD Y SABER

CDMX, MEXICO

